FastAPI Learn Tutorial - User Guide

Body - Nested Models

With **FastAPI**, you can define, validate, document, and use arbitrarily deeply nested models (thanks to Pydantic).

List fields

You can define an attribute to be a subtype. For example, a Python `list`:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list = []


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

This will make `tags` be a list, although it doesn't declare the type of the elements of the list.

List fields with type parameter

But Python has a specific way to declare lists with internal types, or "type parameters":

Declare a `list` with a type parameter

To declare types that have type parameters (internal types), like `list`, `dict`, `tuple`, pass the internal type(s) as "type parameters" using square brackets: `[` and `]`

```python
my_list: list[str]
```

That's all standard Python syntax for type declarations.

Use that same standard syntax for model attributes with internal types.

So, in our example, we can make `tags` be specifically a "list of strings":

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel
```

```python
app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Set types

But then we think about it, and realize that tags shouldn't repeat, they would probably be unique strings.

And Python has a special data type for sets of unique items, the `set`.

Then we can declare `tags` as a set of strings:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

With this, even if you receive a request with duplicate data, it will be converted to a set of unique items.

And whenever you output that data, even if the source had duplicates, it will be output as a set of unique items.

And it will be annotated / documented accordingly too.

Nested Models

Each attribute of a Pydantic model has a type.

But that type can itself be another Pydantic model.

So, you can declare deeply nested JSON "objects" with specific attribute names, types and validations.

All that, arbitrarily nested.

Define a submodel

For example, we can define an `Image` model:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Image(BaseModel):
    url: str
    name: str


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

Use the submodel as a type

And then we can use it as the type of an attribute:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Image(BaseModel):
    url: str
    name: str


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

This would mean that **FastAPI** would expect a body similar to:

```
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
```

```
    "tax": 3.2,
    "tags": ["rock", "metal", "bar"],
    "image": {
        "url": "http://example.com/baz.jpg",
        "name": "The Foo live"
    }
}
```

Again, doing just that declaration, with **FastAPI** you get:

- Editor support (completion, etc.), even for nested models
- Data conversion
- Data validation
- Automatic documentation

Special types and validation

Apart from normal singular types like `str`, `int`, `float`, etc. you can use more complex singular types that inherit from `str`.

To see all the options you have, checkout Pydantic's Type Overview. You will see some examples in the next chapter.

For example, as in the `Image` model we have a `url` field, we can declare it to be an instance of Pydantic's `HttpUrl` instead of a `str`:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()


class Image(BaseModel):
    url: HttpUrl
    name: str


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    image: Image | None = None


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

The string will be checked to be a valid URL, and documented in JSON Schema / OpenAPI as such.

Attributes with lists of submodels

You can also use Pydantic models as subtypes of `list`, `set`, etc.:

Python 3.10+

```python
from fastapi import FastAPI
```

```python
from pydantic import BaseModel, HttpUrl

app = FastAPI()


class Image(BaseModel):
    url: HttpUrl
    name: str


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    images: list[Image] | None = None


@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    results = {"item_id": item_id, "item": item}
    return results
```

This will expect (convert, validate, document, etc.) a JSON body like:

```json
{
    "name": "Foo",
    "description": "The pretender",
    "price": 42.0,
    "tax": 3.2,
    "tags": [
        "rock",
        "metal",
        "bar"
    ],
    "images": [
        {
            "url": "http://example.com/baz.jpg",
            "name": "The Foo live"
        },
        {
            "url": "http://example.com/dave.jpg",
            "name": "The Baz"
        }
    ]
}
```

Info

Notice how the `images` key now has a list of image objects.

Deeply nested models

You can define arbitrarily deeply nested models:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()
```

```python
class Image(BaseModel):
    url: HttpUrl
    name: str


class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: set[str] = set()
    images: list[Image] | None = None


class Offer(BaseModel):
    name: str
    description: str | None = None
    price: float
    items: list[Item]


@app.post("/offers/")
async def create_offer(offer: Offer):
    return offer
```

Info

Notice how `Offer` has a list of `Item`s, which in turn have an optional list of `Image`s

Bodies of pure lists

If the top level value of the JSON body you expect is a JSON `array` (a Python `list`), you can declare the type in the parameter of the function, the same as in Pydantic models:

```python
images: list[Image]
```

as in:

Python 3.10+

```python
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()


class Image(BaseModel):
    url: HttpUrl
    name: str


@app.post("/images/multiple/")
async def create_multiple_images(images: list[Image]):
    return images
```

Editor support everywhere

And you get editor support everywhere.

Even for items inside of lists:

```python
from fastapi import FastAPI
from pydantic import BaseModel, HttpUrl

app = FastAPI()


class Image(BaseModel):
    url: HttpUrl
    name: str


@app.post("/images/multiple/")
async def create_multiple_images(images: list[Image]):
    for image in images:
        image.ur
    return image
```

```
url
update_forward_refs
```

```
url: HttpUrl
```

You couldn't get this kind of editor support if you were working directly with `dict` instead of Pydantic models.

But you don't have to worry about them either, incoming dicts are converted automatically and your output is converted automatically to JSON too.

Bodies of arbitrary `dict`s

You can also declare a body as a `dict` with keys of some type and values of some other type.

This way, you don't have to know beforehand what the valid field/attribute names are (as would be the case with Pydantic models).

This would be useful if you want to receive keys that you don't already know.

Another useful case is when you want to have keys of another type (e.g., `int`).

That's what we are going to see here.

In this case, you would accept any `dict` as long as it has `int` keys with `float` values:

Python 3.10+

```python
from fastapi import FastAPI

app = FastAPI()


@app.post("/index-weights/")
async def create_index_weights(weights: dict[int, float]):
    return weights
```

Tip

Keep in mind that JSON only supports `str` as keys.

But Pydantic has automatic data conversion.

This means that, even though your API clients can only send strings as keys, as long as those strings contain pure integers, Pydantic will convert them and validate them.

And the `dict` you receive as `weights` will actually have `int` keys and `float` values.

Recap

With **FastAPI** you have the maximum flexibility provided by Pydantic models, while keeping your code simple, short and elegant.

But with all the benefits:

- Editor support (completion everywhere!)
- Data conversion (a.k.a. parsing / serialization)
- Data validation
- Schema documentation
- Automatic docs